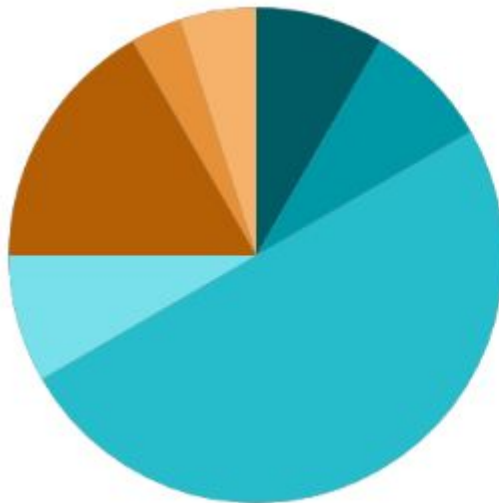


Introduction to Android APIs, Activities, Layouts & Widgets

Ms. Hetal Jethani & Dr. Bhoomi Shah

Department of Computer Science & Engineering

Lecture Structure



	05 Minutes	Outline & Objectives
	05 Minutes	Recap of Previous Lecture
	30 Minutes	Delivery of Actual Content as per Lesson Plan
	05 Minutes	Industry Relevance
	10 Minutes	Interactive Activities
	02 Minutes	Importance of Discipline
	03 Minutes	Recording of Attendance

Outline

1. Introduction to Android & Architecture
2. Development Environment Setup
3. Layouts and UI Design
4. UI Widgets and Input Controls
5. Menus and Navigation

Objectives

1. Understand the Fundamentals of Android Architecture
2. Set Up and Navigate the Android Development Environment
3. Design and Build User Interfaces Using Layouts and Widgets
4. Implement Core Android Features and Menus

Recap of Previous Lecture

- **Mobile communication** enables wireless data and voice exchange, offering portability, mobility, and real-time access through devices like smartphones and tablets.
- **Wireless technologies** have evolved from 1G (analog) to 5G (high-speed, low-latency), with GSM providing a structured framework for global communication.
- **Mobile computing systems** follow a 3-tier architecture—presentation (UI), application (logic), and data (storage)—ensuring efficient, scalable, and secure operation.
- **Ad-hoc and satellite networks** (MANETs and GMSS) support communication in remote, dynamic, or infrastructure-less environments, vital for military, disaster relief, and remote industries.
- **Mobile agents** are intelligent, self-moving programs that perform tasks independently across networks, improving system performance and reducing load.

Introduction to Android APIs

1. Android APIs (Application Programming Interfaces) are essential building blocks for developing Android applications.
2. They provide pre-defined classes and methods to interact with device hardware, software, and services.
3. APIs simplify tasks like accessing the camera, location, sensors, and user interface elements.
4. Developed and maintained by Google as part of the Android SDK (Software Development Kit).
5. APIs evolve with each Android version, offering improved performance, security, and features.

Android Architecture

Android is built on a **multi-layered architecture** that ensures efficiency, security, and performance. The main components include:

1. Linux Kernel

- Core of the Android OS
- Handles hardware abstraction, memory & power management, drivers

2. Hardware Abstraction Layer (HAL)

- Interface between hardware and higher-level APIs
- Enables Android to support diverse hardware

3. Android Runtime (ART) & Libraries

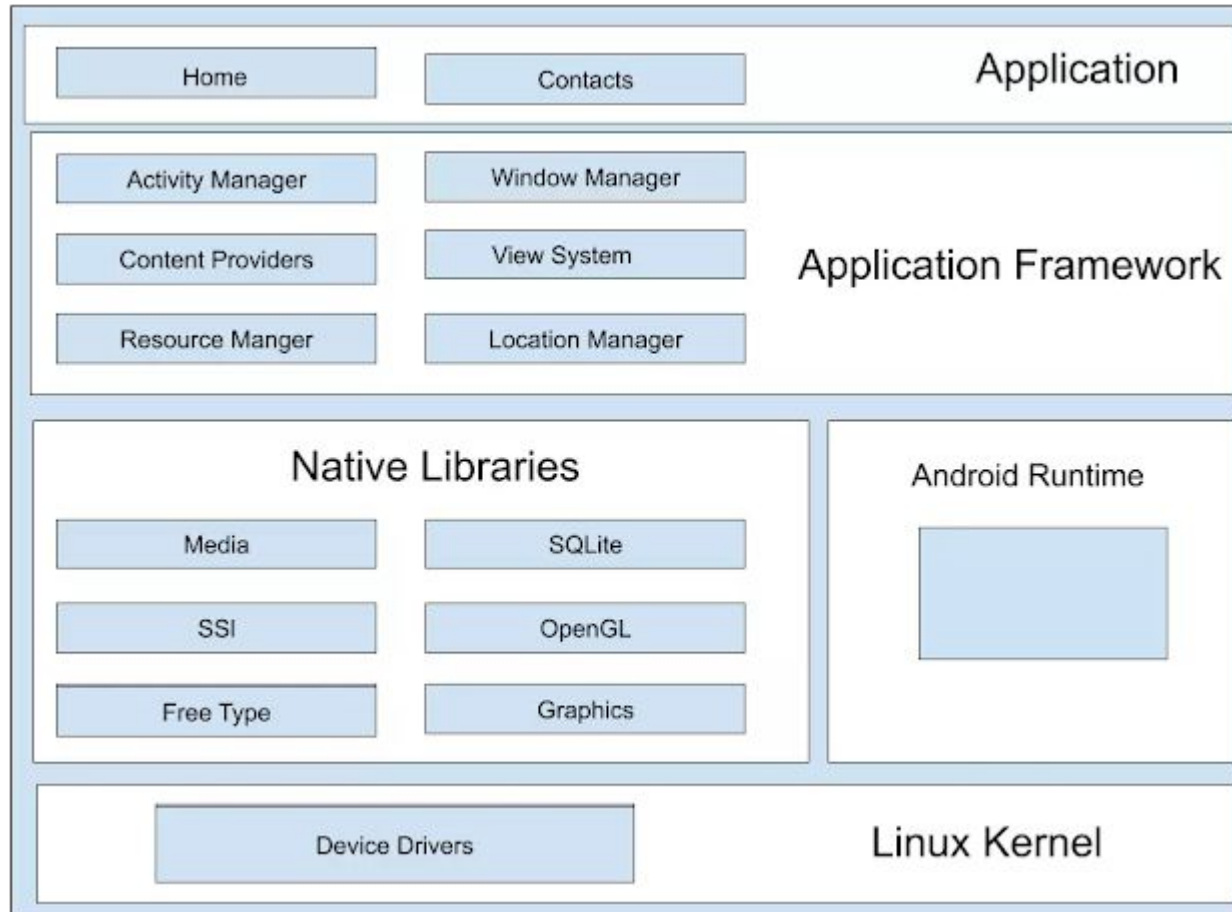
- ART: Executes app bytecode with improved performance
- Core libraries: Provide Java-based functionality

4. Application Framework

- High-level building blocks (Activities, Services, Broadcast Receivers, Content Providers)
- Allows reuse of components across apps

5. Applications

- User-installed and pre-installed apps
- Built using the Application Framework



Application Framework

- Provides high-level APIs for building Android apps.
- Manages the **interaction between apps and system services**.
- Allows developers to access essential components and services.

Key Components:

1. **Activity Manager** – Manages the activity lifecycle and app navigation.
2. **Content Providers** – Share data between apps securely.
3. **Resource Manager** – Handles external resources like layouts, strings, and images.
4. **View System** – UI components like buttons, text fields, layouts.
5. **Package Manager** – Manages app installation and updates.
6. **Notification Manager** – Displays alerts to users.
7. **Location Manager** – Accesses GPS and network location.

The Application Components

- Android apps are built using four key components that define how the app behaves and interacts.

1. Activity

- Represents a single screen with a user interface.
- Manages UI interactions.
- Example: Login screen, home screen.

2. Service

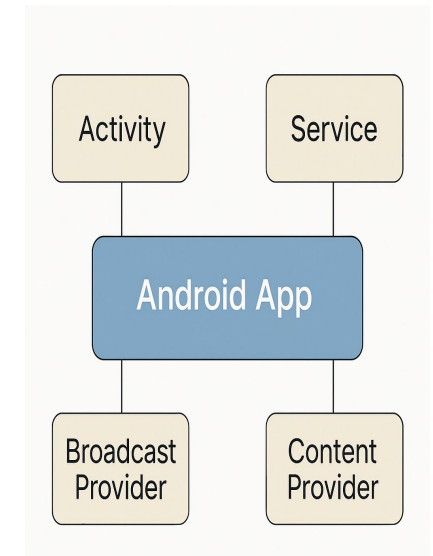
- Performs background tasks without a UI.
- Runs even when the app is not in the foreground.
- Example: Music playback, syncing data.

3. Broadcast Receiver

- Listens for system-wide or app-specific events.
- Responds to broadcast messages (e.g., low battery, network change).

4. Content Provider

- Manages shared app data.
- Allows apps to access and modify data securely (contacts, media files, etc.)



The Manifest File

- **AndroidManifest.xml** is a critical file in every Android app.
- It provides essential information to the Android system about the app.
- Acts as a **bridge between the application and the Android OS**.

Key Roles of the Manifest File:

1. Declare Application Components

1. Lists all activities, services, broadcast receivers, and content providers.

2. Permissions

1. Requests permissions (e.g., internet, camera, location) required by the app.

3. App Metadata

1. Specifies app name, icon, theme, version, and minimum SDK.

4. Intent Filters

1. Defines how components respond to system or user actions.

5. Hardware/Software Features

1. Declares device features the app requires (e.g., GPS, Bluetooth).

```
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
    package="com.example.myapp">

    <!-- Permission declaration -->
    <uses-permission android:name="android.permission.INTERNET"/>

    <!-- Minimum and target SDK -->
    <uses-sdk android:minSdkVersion="21"
        android:targetSdkVersion="34" />

    <application
        android:allowBackup="true"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:theme="@style/AppTheme">

        <!-- Declare Main Activity -->
        <activity android:name=".MainActivity">
            <intent-filter>
                <action android:name="android.intent.action.MAIN"/>
                <category android:name="android.intent.category.LAUNCHER"/>
            </intent-filter>
        </activity>

        <!-- Declare a Service -->
        <service android:name=".MyBackgroundService" />

    </application>
</manifest>
```

Setting Up the Android Development Environment

1. Download Android Studio

- Visit the official site: developer.android.com/studio
- Choose your OS (Windows, macOS, or Linux).
- Click "Download Android Studio" and accept the terms.

2. Install Android Studio

- Run the downloaded installer.
 - Install Android SDK
 - Android Virtual Device (AVD)
 - SDK Tools and Emulator

3. First Launch

- Open Android Studio
- Configure system settings (theme, SDK path)
- Start a new project or import an existing one

4. Verify Installation

- Run a sample "Hello World" app on an emulator or real device.

Exploring the Development Environment

1. Project Structure

- Android app organized into **modules**, **manifests**, **Java/Kotlin source code**, and **resources**.
- Common folders: `java/`, `res/`, `manifest/`.

2. Code Editor

- Smart code completion, syntax highlighting, and real-time error checking.
- Integrated with **Jetpack libraries** and **Gradle build system**.

3. Layout Editor

- Design UI visually with **drag-and-drop** tools.

4. Emulator & Device Manager

- Test apps on virtual Android devices.
- Supports camera, GPS, sensors, and more.

5. Logcat & Debugger

- View system logs and debug your app in real-time.

6. Build & Run Toolbar

- Compile, run, and deploy apps directly from the IDE.

Developing and Executing the First Android Application

1. Start a New Project

- Open Android Studio → *New Project*
- Choose "**Empty Activity**" template
- Name your app and select language (Java/Kotlin)

2. Configure Project

- Set package name, save location, and minimum SDK
- Click "Finish" to generate project structure

3. Design the UI

- Open activity_main.xml in the res/layout folder
- Add a simple TextView

4. Write Code (MainActivity.java/Kotlin)

- Logic already generated to display the layout

5. Run the App

- Select a device/emulator from the toolbar
- Click **Run** (▶) to build and deploy the app
- Output: "Hello World!" on screen

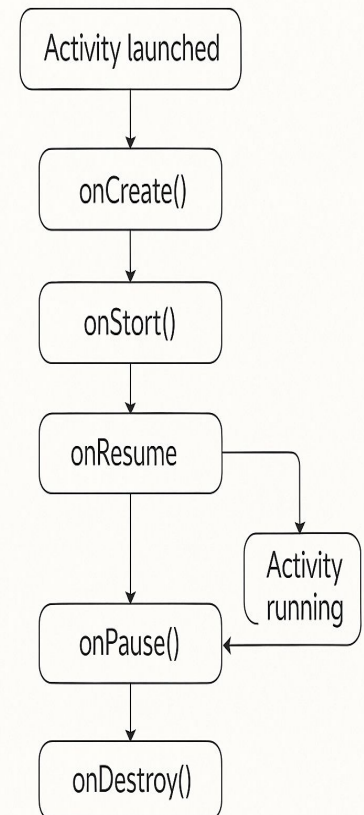
Introduction to Activity & its life cycle

1. What is an Activity?

- An Activity is a single, focused screen with a user interface.
- Each screen in an app is typically one Activity.
- The **MainActivity** is the entry point of most apps.

2. Activity Lifecycle

- Android manages Activities through a set of lifecycle methods:
 - `onCreate()` – Initialization (UI setup)
 - `onStart()`, `onResume()` – UI becomes visible and interactive
 - `onPause()`, `onStop()` – Activity goes into background
 - `onDestroy()` – Cleanup before activity is destroyed



Working with Activities

3. Starting Another Activity

java

```
Intent intent = new Intent(this, SecondActivity.class);  
startActivity(intent);
```

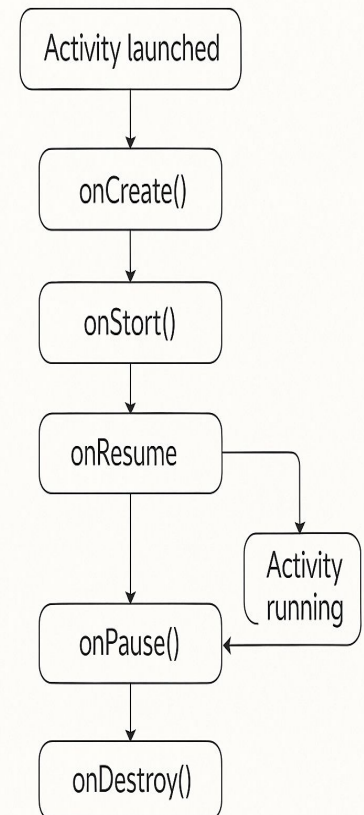
4. Navigating Between Activities

- Use **Intents** to switch and pass data between screens.

5. Declaring Activities in Manifest

xml

```
<activity android:name=".SecondActivity" />
```



The LinearLayout Layout

1. What is LinearLayout?

- A layout that arranges its children **in a single row or column**.
- Orientation can be **vertical** or **horizontal**.

2. Common Attributes

- `android:orientation="vertical"` or `"horizontal"` – Sets layout direction
- `android:layout_width / layout_height` – Defines size of children
- `android:layout_weight` – Distributes space proportionally
- `android:gravity` – Aligns children inside the layout

3. Example: Vertical LinearLayout XML code

```
<LinearLayout android:layout_width="match_parent"
android:layout_height="wrap_content"
android:orientation="vertical"> <TextView android:text="Welcome" />
<Button android:text="Click Me" /> </LinearLayout>
```

4. When to Use

- Use for simple, ordered layouts like login forms, menus, or vertical lists.

The Relative Layout

1. What is RelativeLayout?

- A layout where child views are **positioned relative to each other or the parent**.
- Offers flexible UI design without nesting multiple layouts.

2. Common Attributes

- `android:layout_alignParentTop="true"` – Aligns view to top of parent
- `android:layout_below="@id/anotherView"` – Places view below another
- `android:layout_centerHorizontal="true"` – Centers view horizontally
- `android:layout_toRightOf="@id/view1"` – Positions to the right of another view

The Relative Layout Continued...

3. Example XML code

```
<RelativeLayout android:layout_width="match_parent"
android:layout_height="match_parent">
<TextView android:id="@+id/title" android:text="Hello"
android:layout_alignParentTop="true" />
<Button android:text="Click" android:layout_below="@id/title" />
</RelativeLayout>
```

4. When to Use

- Ideal for dynamic UIs where elements adjust based on others' positions.

The ScrollView Layout

1. What is ScrollView?

- A **layout container** that allows users to scroll content **vertically** (or horizontally with `HorizontalScrollView`).
- Useful when content exceeds the screen size.

2. Key Rules

- Can **contain only one direct child** (usually a `LinearLayout` or `ConstraintLayout`).
- Child layout should have `wrap_content` height and width to allow scrolling.

The ScrollView Layout Continued.....

3. Common Attributes

- `android:fillViewport="true"` – Fills the screen if content is smaller
- `android:scrollbars="vertical"` – Enables scroll indicator

4. Example

xml

```
<ScrollView android:layout_width="match_parent"
android:layout_height="match_parent">
<LinearLayout android:orientation="vertical"
android:layout_width="match_parent" android:layout_height=
"wrap_content"> <!-- Long list of items --> <TextView
android:text="Item 1" />
<TextView android:text="Item 2" /> <!-- ... --> </LinearLayout>
</ScrollView>
```

5. When to Use

- For forms, long content, FAQs, or vertically stacked items.

The TableLayout Layout

1. What is TableLayout?

- Arranges UI components into **rows and columns**, similar to a spreadsheet.
- Each row is defined using a TableRow element.

2. Structure & Behavior

- Doesn't display borders between cells by default.
- Each TableRow can contain multiple views (usually TextView, EditText, Button, etc.).
- Columns are aligned across rows based on their order.

3. Common Attributes

- android:stretchColumns="*", "1,2" – Stretches specified columns to fill space
- android:shrinkColumns – Allows columns to shrink
- android:collapseColumns – Hides columns

The Table Layout Continued..

4. Example

xml

```
<TableLayout android:layout_width="match_parent" android:layout_height="wrap_content">
<TableRow> <TextView android:text="Name" /> <EditText android:hint="Enter Name" />
</TableRow> <TableRow> <TextView android:text="Email" /> <EditText android:hint="Enter
Email" />
</TableRow> </TableLayout>
```

5. When to Use

- Best for form-like layouts or grids of aligned text fields and labels.

The Frame Layout

1. What is FrameLayout?

- A simple layout that **displays one view at a time**, or **stacks views on top of each other**.
- Ideal for overlays, banners, and background images.

2. Behavior & Characteristics

- Child views are positioned in the **top-left corner** by default.
- Views can **overlap**—the last added view appears on top.

3. Common Use Cases

- Splash screens
- Loading overlays
- Media players (e.g., video with controls on top)
- Background image behind content

The Frame Layout Continued

4. Example

xml

```
<FrameLayout android:layout_width="match_parent" android:layout_height="match_parent">  
<ImageView android:src="@drawable/bg_image" android:layout_width="match_parent"  
android:layout_height="match_parent" /> <TextView android:text="Welcome"  
android:layout_gravity="center" />  
</FrameLayout>
```

5. When to Use

- When layering is needed and layout complexity is minimal.

Using the TextView

1. What is TextView?

- A UI widget used to **display text** to the user.
- Supports **plain text**, **HTML**, **links**, and **formatting**.

2. Common Attributes

- `android:text` – Sets the text content
- `android:textSize` – Font size (sp units)
- `android:textColor` – Text color
- `android:gravity` – Text alignment
- `android:padding` – Inner spacing around text

Using the TextView Continued....

3. Basic Example

xml

```
<TextView android:layout_width="wrap_content" android:layout_height="wrap_content"
android:text="Welcome to Android" android:textSize="20sp" android:textColor="#008000"
android:gravity="center" />
```

4. Usage Tips

- Use for labels, messages, or any read-only text.
- Combine with SpannableString for rich text styling.

Using the EditText View

1. What is EditText?

- A UI widget that lets the user **enter and edit text**.
- Commonly used for forms like login, registration, search, etc.

2. Input Types

- text – General text
- textEmailAddress – Email input
- textPassword – Password fields
- number, phone, date, etc.

3. Common Attributes

- android:hint – Shows placeholder text
- android:inputType – Specifies expected keyboard
- android:maxLines, android:lines – Controls input space
- android:ems – Width in character units

Using the EditText View

4. Example

xml

```
<EditText android:layout_width="match_parent" android:layout_height="wrap_content"  
android:hint="Enter your name" android:inputType="textPersonName" />
```

5. Usage Tips

- Always use `inputType` to enhance user experience
- For passwords, use `inputType="textPassword"` and `transformationMethod`

Using the Button View

1. What is Button?

- A clickable UI element used to **perform actions** when tapped by the user.
- Commonly used for submitting forms, navigating screens, or triggering events.

2. Common Attributes

- `android:text` – Label shown on the button
- `android:onClick` – Method to execute on click
- `android:background` – Set background color or image
- `android:enabled` – Enable/disable interaction
- `android:textColor` – Color of the button text

3. Example XML code

```
<Button android:layout_width="wrap_content" android:layout_height="wrap_content"  
    android:text="Login"  
    android:onClick="handleLogin" />
```

4. Handling Clicks in Java

```
public void handleLogin(View view) { // Code to handle button click }
```

Using the RadioButton

1. What is RadioButton?

- A UI element that allows the user to **select one option** from a group.
- Always used inside a **RadioGroup** to manage selection behavior.

2. Common Attributes

- android:text – Label for the option
- android:checked – Sets default selected button
- android:onClick – Action on selection
- android:id – Needed for grouping and referencing

3. Example: Gender Selection

xml

```
<RadioGroup android:layout_width="wrap_content"
android:layout_height="wrap_content">
<RadioButton android:id="@+id/radioMale" android:text="Male" /> <RadioButton
android:id="@+id/radioFemale"
android:text="Female" /> </RadioGroup>
```


Using the RadioButton

4. Handling Selection in Code

java

```
RadioGroup group = findViewById(R.id.radioGroup);  
int selectedId = group.getCheckedRadioButtonId();  
RadioButton selected = findViewById(selectedId);  
String choice = selected.getText().toString();
```

5. Usage Tips

- Only one RadioButton can be selected in a group
- Ideal for mutually exclusive options like Yes/No, gender, payment method

Using the CheckBox

1. What is CheckBox?

- A UI widget that lets users **select one or more options** from a list.
- Each CheckBox works **independently** (unlike RadioButtons).

2. Common Attributes

- android:text – Label for the checkbox
- android:checked – Sets default checked state
- android:onClick – Action triggered on click
- android:id – Used for referencing in code

3. Example: Select Interests

xml

```
<CheckBox android:id="@+id/checkbox_music" android:text="Music" />
```

```
<CheckBox android:id="@+id/checkbox_sports" android:text="Sports" />
```

Using the CheckBox Continued..

4. Handling State in Code

```
java  
CheckBox music = findViewById(R.id.checkbox_music); if (music.isChecked()) { //  
User selected Music }
```

5. Usage Tips

- Great for preference settings, multi-select lists, and filters
- Use a **clear label** and **group visually** when there are many options

Using the ImageButton

1. What is ImageButton?

- A UI component that behaves like a button but **displays an image instead of text**.
- Useful for toolbar icons, social buttons, media controls, etc.

2. Common Attributes

- `android:src` – Sets the image to display
- `android:contentDescription` – Improves accessibility (required!)
- `android:background` – Optional background styling
- `android:onClick` – Handles click action

3. Example

xml

```
<ImageButton android:id="@+id/btn_settings" android:layout_width="wrap_content"
android:layout_height="wrap_content" android:src="@drawable/ic_settings"
android:contentDescription="Settings" android:onClick="openSettings" />
```

Using the RatingBar

1. What is RatingBar?

- A UI widget that allows users to **rate content using stars**.
- Commonly used in feedback forms, reviews, or product ratings.

2. Common Attributes

- `android:numStars` – Total number of stars
- `android:rating` – Default selected rating
- `android:stepSize` – Increments (e.g., 0.5 for half-stars)
- `android:isIndicator` – Set to true for read-only display

3. Example

xml

```
<RatingBar android:id="@+id/ratingBar" android:layout_width="wrap_content"  
android:layout_height="wrap_content" android:numStars="5" android:stepSize="1.0"  
android:rating="3" />
```

The Options Menu

1. What is the Options Menu?

- A **primary menu** in Android that appears when the user taps the **menu button** or three-dot icon.
- Commonly used for settings, search, help, etc.

2. Defining the Menu (XML)

- Create a file in `res/menu/options_menu.xml`:

xml

```
<menu xmlns:android="http://schemas.android.com/apk/res/android">  
<item android:id="@+id/menu_settings" android:title="Settings" />  
<item android:id="@+id/menu_help" android:title="Help" /> </menu>
```

3. Inflating the Menu in Activity

java

```
@Override public boolean onCreateOptionsMenu(Menu menu) {  
    getMenuInflater().inflate(R.menu.options_menu, menu); return true; }
```

The Context Menu

1. What is a Context Menu?

- A **floating menu** that appears when the user **long-presses a view**.
- Shows actions specific to the selected item (e.g., edit, delete).

2. When to Use

- For actions related to list items, images, or UI elements
- Useful in apps with dynamic content (e.g., messaging, file browsers)

3. Defining the Menu (XML)

xml

```
<menu xmlns:android="http://schemas.android.com/apk/res/android">  
<item android:id="@+id/edit" android:title="Edit"/>  
<item android:id="@+id/delete" android:title="Delete"/> </menu>
```

Industry Relevance

How **API,Activities,Layouts** and **Widgets** useful for the industry:

Companies like **Zomato, Ola, and Paytm** rely on Android APIs to access location, send notifications, etc.

Activities are used in banking apps, fitness trackers, and educational platforms.

Widgets are the **building blocks of UI/UX** in Android applications.

Industry Relevance Continued....

How Mobile Computing is useful for the industry:

Summary

Component	Industry Relevance
API	Enables complex, real-world functionality
Activity	Controls app flow and user interaction
Layout	Delivers professional, responsive design
Widget	Enhances interactivity and usability

Interactive Activity-Mobile Computing :

A real-world scenario to design, such as:

A login screen

A food ordering app

A fitness tracking dashboard

A weather app home screen

GSFCU & Values : Cultivating a Positive Mindset

Components of a Positive Mindset



Made with Napkin

Recommended Books

